Image and Volume Processing Report

Advanced Programming

Jiayu Wu, Oliver Whitehead, Xinying Yang, Julia Williams, Hao Liu, Anton Belov, Yuhang Dai

March 20, 2026

1 Task Allocation

The development of this project was carried out through a combination of collaborative and pair-based work. While the overall software architecture and codebase structure were designed collectively by the entire group, specific components were delegated to smaller teams to ensure efficiency and attention to detail. Oliver created the `Image` and `Volume` classes, as well as the algorithms for projection and slicing operations. Jaiyu and Xinying implemented the simple filters and assisted with creating scaling plots. The 2D and 3D blur filters were designed by Julia and Yuhang. Finally, the edge detection, sharpening, and embossing filters were designed by Anton and Hao. Overall, the project was a coordinated team effort, combining shared architectural structure with specific contributions from smaller groups. This approach ensured both uniformity across the codebase and methodical design of individual components.

2 Code Design

To ensure consistency and efficiency, the image processing pipeline is designed using an object-oriented architecture. A common abstract `Filter` base class defines a standard interface through virtual `apply` methods for both 2D `Image` and 3D `Volume` objects. The `Image` class represents 2D data with support for multiple channels, while the `Volume` class extends this concept to 3D data, storing voxel information across depth as well as height and width. Each specific filter is implemented as a derived class that provides its own algorithmic implementation of the `apply` function.

The `Image` and `Volume` classes also contain a set of functions for performing efficient per-pixel/per-voxel operations or color transformations. The `getPixel()` and `setPixel()` provide direct access to the underlying 1D data array. These functions handle the conversion from 2D coordinates to linear memory indices, allowing filter implementations to work with intuitive (x, y, c) coordinates while avoiding repeated index calculations. Additionally, `getPixel()` handles out-of-bounds accesses by wrapping coordinates outside the image domain. This methodology is extended to voxels in the `Volume` class. A decision to move the pixel getter and setter methods into `Image.h` and marking them as `inline` allowed the compiler to perform optimisations, such as loop unrolling and auto-vectorization, while eliminating function call overhead.

This design enables polymorphism across filters such as Gaussian and median blur, which can operate on both 2D images and 3D volumes. By implementing the appropriate `apply` methods, these filters can be used interchangeably regardless of input dimensionality. When accessed through a base class reference, the correct implementation is selected at runtime, ensuring flexibility while maintaining a unified interface. This approach exhibits modularity, making it simple to add new filters or extend to various input types without modifying existing components.

3 Simple Filters

Basic image processing operations were implemented to manipulate intensity, contrast, and noise characteristics while maintaining a streamlined design. To optimise simple filters, a specialised pixel access method `getPixelSimple` is used, which directly retrieves the specified pixel value without performing boundary checks. Since simple filters operate only on valid pixel regions, clamping is unnecessary, allowing the function to avoid conditional overhead.

4 Blur Filters

Each filter - box blur, Gaussian blur, and median blur - takes in a user specified kernel size and utilizes a different strategy to combine neighbouring values, resulting in their different

visual effects and performance characteristics. The Gaussian and median filters are designed to operate on 2D images and 3D volumes, demonstrating flexibility and scalability within a modular framework.

The box blur is the simplest method, replacing each pixel with the average of its surrounding values. To improve efficiency, it is implemented using a two-pass sliding window approach rather than recomputing the full kernel at each pixel. Running sums are computed and updated incrementally across rows and columns, reducing the computational cost from $O(k^2)$ per pixel to $O(k)$, significantly improving performance.

The Gaussian blur similarly uses a separable approach by first constructing a 1D Gaussian kernel, which is then applied in multiple passes. This method is also extended to 3D volumetric data by adding an additional pass in the depth direction. This method exhibits the same computational cost per pixel or voxel, O(k), as our sliding window box blur approach.

The median blur filter is implemented using a histogram-based sliding window approach for both 2D images and 3D volumes. Rather than sorting all neighbouring values at every pixel or voxel, the algorithm creates a histogram of intensity counts over the current kernel window and extracts the median by scanning cumulatively until the middle element is reached. As the window moves along the x-direction, the histogram is updated incrementally by removing values that leave the window and adding those that enter, which avoids rebuilding it from scratch each time. This streamlined implementation maintains the noise-reduction strength of the median filter, and the same strategy is once again extended from images to volumetric data by including the additional depth dimension.

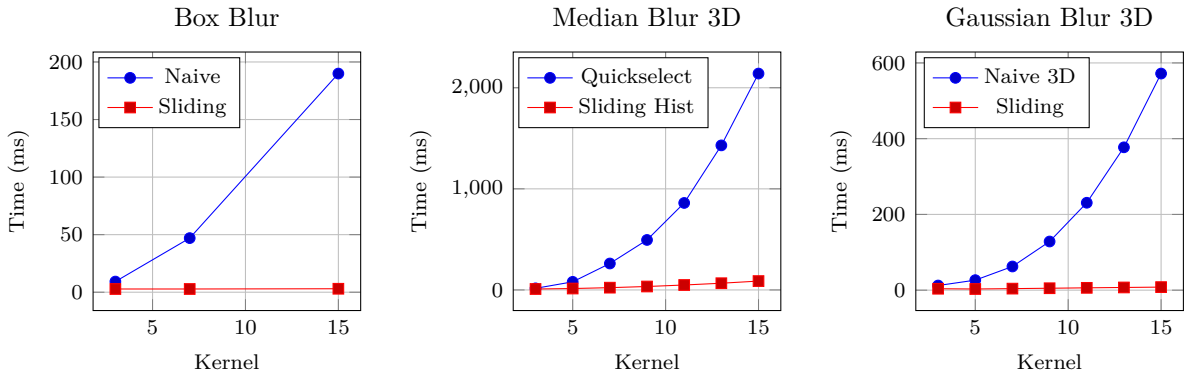


Figure 1: Performance comparison between baseline and optimised implementations of box, median, and Gaussian blur filters. The results highlight the reduced computational complexity of sliding window and histogram-based approaches, particularly for larger kernel sizes.

5 Convolutional Filters

The convolutional filters are structured around the `ConvFilter` base class, which implements the shared logic for kernel storage, per-channel convolution, result normalisation, and writing filtered values back into the image. It has one constructor that accepts a single kernel, and another that accepts two kernels, letting multiple filters reuse the same inherited convolution pipeline while specialising it through construction and overridden behaviour.

Both the image pixel buffer and the convolution kernels are stored as contiguous one-dimensional arrays rather than two-dimensional vectors. Because modern CPUs fetch memory in cache lines rather than individual bytes, this approach ensures that sequential reads during the convolution inner loop access memory located in L1 or L2 cache, dramatically reducing cache misses compared with a pointer-to-pointer 2D structure where each row could be allocated at an arbitrary, non-adjacent memory address.

A key design decision was to separate generic convolution mechanics from filter-specific behaviour. This avoids duplicating the same nested-loop convolution code across multiple filters and makes the system easier to extend: to add a new filter, one only needs to define the kernel

and override `apply()` when extra behaviour is required, such as channel conversion or output normalisation. The main interaction between classes is as follows:

- The `Image` class owns the raw pixel buffer and provides both safe and unsafe pixel/data access methods.
- `ConvFilter` reads image data, performs convolution, stores intermediate floating-point results, and writes the final clamped values back to the image.
- Derived filter classes - `SharpenFilter`, `SobelFilter`, `PrewittFilter`, `ScharrFilter`, `RobertsCrossFilter` and `EmbossFilter` - configure `ConvFilter` by passing either one kernel or two kernels to the base constructor.

The base `ConvFilter::apply()` performs generic convolution and output writing, while derived classes override `apply()` to add extra steps such as converting RGB images to grayscale, computing directional responses, calculating a theoretical maximum, normalising the result, and then writing the processed output. Additionally, the base class hides the internal details behind helper methods such as `computeResults()`, `convolveChannel()`, `normalise()`, and `writeResults()`. This improves encapsulation and makes the code easier to test and reason about, because the responsibilities of each stage are clearly separated.

One important decision was to store intermediate convolution results as floating-point values rather than writing directly to the image buffer. This is necessary for preserving precision until the final clamping stage and preventing already-modified output pixels from affecting later reads. Additionally, `writeResults()` obtains a writable raw pointer using `getDataUnsafe()` and increments it sequentially, which reduces indexing overhead during the final copy.

A second design decision was to distinguish between interior pixels and border pixels. The implementation uses two different pixel-access paths: a fast path for interior pixels using `getPixelSimple()` with no boundary checks, and a slower edge path using `getPixel()` with clamping for effectively replicating the "extend" functionality. Since in a large image most pixels are interior pixels, removing repeated bounds checking from the inner region reduces overhead.

A third decision was to support fused two-kernel processing inside the convolution pipeline, instead of convolving the image twice in separate passes. When two kernels are present, the code computes both directional responses during the same traversal of the kernel window and combines them using the Euclidean magnitude. Because the results of the convolution are not guaranteed to be within the image pixel range, the `normalise()` method calculates a theoretical maximum response of each filter and rescales the output to the 0-255 range.

Additionally, to reduce both arithmetic overhead and memory traffic we removed some extra arithmetic steps from the hottest part of the algorithm by applying them to the kernels at creation. For example, in `EmbossFilter`, the `strength` parameter is applied once when the object is created by scaling each kernel coefficient directly, so the convolution code can later use the updated kernel immediately instead of multiplying the result of every pixel by `strength` during execution. In `SharpenFilter`, increasing the centre coefficient to 5 effectively folds the original image contribution into the convolution kernel itself, so the sharpened output $(I + G)$ is produced in a single pass rather than first computing the filtered response and then adding it back to the original image in a separate operation.

The time complexity of the convolution stage is approximately $O(W \times H \times K_w \times K_h \times C)$, where W and H are image width and height, K_w and K_h are kernel dimensions, and C is the number of channels. This follows directly from the nested loops in `convolveChannel()`, where each output pixel is computed by visiting every element of the kernel window.

6 Projections and Slices

We further extend the functionality of our package for 3D Volumes with projection and slicing operations. These are used to extract 2D representations from 3D volumetric data with various techniques. Our slicing operation selects individual cross-sections of the volume along a chosen

axis, while the projection methods collapse information across multiple slices to produce a single image. We implement a variety of projection methods, each summarising voxel values along the depth dimension in different ways. These operations provide complementary views of the data, enabling analysis of volumetric structures.

The projection methods include maximum (MIP), minimum (MinIP), and mean (meanIP) intensity projections, which convert a 3D volume into a 2D image along the z-axis. A helper function processes user-defined slice bounds by converting 1-based indices to valid 0-based ranges and ensuring they lie within bounds. Each method combines voxel intensities across the selected depth using direct 1D memory indexing and a precomputed slice stride, computing the maximum, minimum, or average value for each pixel.

The Median Intensity Projection (MedianIP) was initially implemented using Quickselect, but this approach suffered from dynamic memory overhead, recursion costs, and potential $O(N^2)$ worst-case behaviour. It was therefore optimised using a histogram-based method that leverages the limited 8-bit intensity range (0–255), enabling median computation in linear time $O(N)$ with no recursion or additional allocations, significantly improving performance.

The slicing implementation extracts 2D cross-sectional views from the 3D volume along specified axes. After converting user input to valid indices, it fixes one dimension and iterates over the remaining two to construct a new `Image`. The z-axis is inverted during mapping to ensure a correct visual orientation, allowing for a 2D visualisation of volumetric data.

7 Testing

Our filters were tested using predefined image matrices with known pixel values. The expected results were analytically calculated and then compared to our filtered outputs to check correctness. Outputs were also visually inspected to ensure that each filter produced the expected qualitative effect. In addition to this, we performed general testing to ensure our filter outputs are correctly formatted, including validating image dimensions and colour channels.

8 Performance

Figure 2 shows the runtime and memory usage of the 2D image filters as input size increases. Most filters exhibit linear runtime scaling with respect to the number of pixels, consistent with their per-pixel computations. Simple operations such as brightness adjustment and thresholding are the most efficient, while more complex filters like Gaussian and median blur incur higher costs due to calculations involving surrounding pixels.

Memory usage also scales linearly with input size and is largely similar across filters, indicating that it is dominated by image storage rather than the filtering operations themselves. Overall, the results demonstrate that the implementations are efficient and scale well, with performance primarily determined by the complexity of each filter.

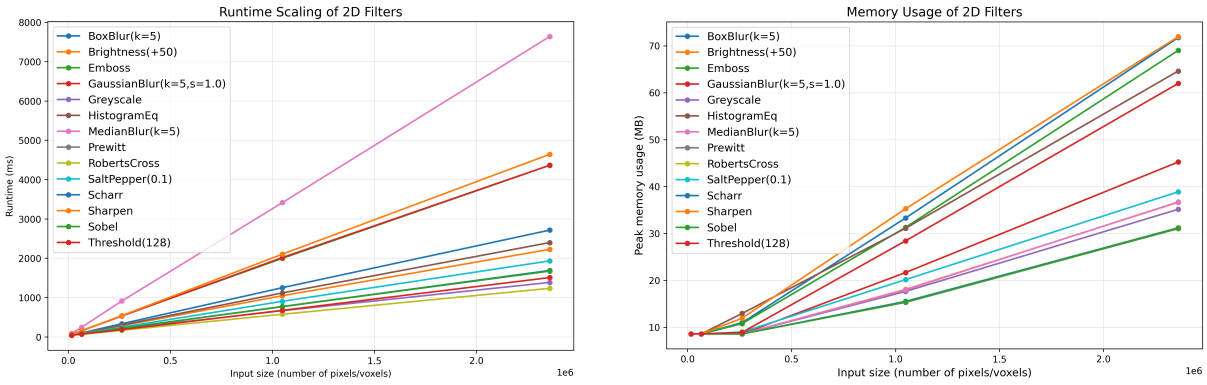


Figure 2: Benchmark plots of all 2D image filters for various image sizes showing the runtime scaling (left) and peak memory usage (right).